PROJECT-BASED LEARNING (PBL) REPORT

DATABASE MANAGEMENT SYSTEMS

DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM FOR COURIER PARCEL BOOKING AND TRACKING MANAGEMENT SYSTEM



Program: B. Tech CSE AIML

Section: LEOPARDS

Faculty: DR. NAGARAJU

Submitted by

P.SAI RITHVIK YADAV

25WU0102204

ACKNOWLEGEMENT

I express my sincere gratitude to Woxsen University, School of Technology, for providing me with the opportunity to undertake this Project-Based Learning (PBL) activity as part of the Database Management Systems (DBMS) course. I thank my course faculty, DR. NAGARAJU, for guidance, suggestions, and support throughout the design and development of this project. This work has helped me apply database concepts including ER modelling, relational schema design, normalization, SQL, integrity constraints, and database application development.

UNDERTAKING

I hereby declare that the project entitled “Design and Implementation of a Database Management System for Courier Parcel Booking and Tracking Management System” is my work carried out as part of the DBMS Project-Based Learning activity under the guidance of the course faculty. I acknowledge the sources, tools, and resources consulted. I am responsible for the correctness and authenticity of the submitted report, database scripts, application source code, results, and supporting materials.

Name: SAI RITHVIK Roll Number: 25WU0102204 Program: B.Tech CSE AIML

TABLE OF CONTENTS

- Abstract
- Keywords
- List of Tables
- List of Figures
- List of Acronyms
- 1. Problem Statement, Solution Strategy and ER Diagram
- 2. Logical Database Design
- 3. Application Development Using Logical Database Design
- 4. Conclusion
- 5. References
- 6. Appendices

Note: Update page numbers and generate Word's automatic table of contents after final formatting.

LIST OF TABLES

Table No.	Title
1	Scope and Assumptions
2	Proposed Solution Strategy
3	Functional Requirements
4	Entity and Relationship Summary
5	Relational Schema
6	Keys and Integrity Constraints
7	Normalization Status up to 3NF
8	Data Dictionary
9	Database Implementation Environment
10	Application Development Environment
11	Application Module-to-Database Mapping
12	CRUD Operations
13	Business Rules, Validation and Error Handling
14	Reports and Database Operations
15	Test Cases

LIST OF FIGURES

Figure No.	Title
1	Entity-Relationship Diagram of the Proposed System
2	Parcel Booking Screen — insert actual screenshot
3	Parcel Tracking Screen — insert actual screenshot
4	Delivery Update Screen — insert actual screenshot
5	Reports Screen — insert actual screenshot

LIST OF ACRONYMS

Acronym	Expansion
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
ERD	Entity-Relationship Diagram
FK	Foreign Key
PK	Primary Key
SQL	Structured Query Language
3NF	Third Normal Form

ABSTRACT

Courier organizations manage parcel booking, routing, tracking, delivery attempts, and payments across branches and hubs. When these activities are recorded in disconnected registers or spreadsheets, staff may face inconsistent records, slow tracking, duplicate entries, and difficulty preparing operational reports. The objective of this project is to design and implement a relational database application that organizes the main information required for courier parcel management. The proposed design identifies customers, branches, hubs, service types, bookings, parcels, routes, scan events, vehicles, delivery agents, delivery attempts, charges, and payments as core entities. The database design is represented through an entity-relationship model and mapped to relational tables with primary keys, foreign keys, uniqueness rules, validation

constraints, and normalization up to Third Normal Form. A Python application using Streamlit is proposed as the user interface, with SQLite as the relational database for the local prototype. The application workflow covers parcel booking, charge calculation, tracking lookup, scan updates, delivery status changes, and report generation. SQL queries support searching, joining, grouping, and summarizing records. The resulting system provides a practical demonstration of database design and application connectivity; implementation evidence and test outcomes must be added after execution.

Keywords: Courier Management, Relational Database, ER Diagram, Normalization, SQL, Python

CHAPTER 1

PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

Courier firms need to maintain consistent information from the moment a parcel is booked until it is delivered. The process involves customer details, parcel attributes, service selection, charges, routing through branches or hubs, scan events, delivery-agent assignment, delivery attempts, and payment records. If these activities are managed through manual registers or separate files, records can become duplicated or inconsistent. Staff may need to search multiple sources to identify the latest parcel status, determine where a delay occurred, or calculate branch-level workload and revenue.

The project addresses this problem by designing a centralized relational database and a small Python application. The system is intended to store related records using keys and constraints, provide tracking search, record parcel movement, and produce useful reports. The prototype is a local academic demonstration and does not claim live GPS tracking or integration with external courier networks.

1.2 Objectives

- Design an ER model representing the main courier entities, relationships, and cardinalities.
- Convert the ER model into a relational schema and document normalization up to 3NF.
- Implement tables with primary keys, foreign keys, uniqueness rules, and suitable validation constraints.
- Develop a Python application for booking, charge calculation, tracking updates, and reporting.
- Validate the database and application using positive and negative test cases.

1.3 Scope and Assumptions

Item	Description
In Scope	Customer and parcel booking records; service and charge calculation; tracking scan history; branch/hub and route concepts; agent and delivery attempt records; payments;

	SQL reports; local Python application.
Out of Scope / Exclusions	Live GPS feeds, third-party courier APIs, production payment gateway, SMS/email notifications, multi-tenant deployment, and production-grade authentication.
Assumptions	Each booking has a unique tracking number; weights and monetary amounts must be positive; scan events are timestamped; only valid status transitions are permitted; sample records are synthetic.

1.4 Proposed Solution Strategy

Solution Element	Project-Specific Description
Primary Users / Roles	Booking operator, tracking/operations staff, delivery coordinator, and administrator.
Major Application Modules	Parcel booking, charge calculation, tracking lookup, scan entry, delivery update, payment recording, and reports.
Proposed Workflow	Capture booking → validate inputs → calculate charge → save booking and parcel → record scans → assign route/agent → update delivery outcome → report.
Important Business Rules	Unique tracking number; positive weight and charge; valid foreign keys; chronological scan events; no ordinary scans after delivery; payment total cannot exceed amount due.
Expected Reports / Outputs	Current parcel status, scan history, delayed shipments, branch load, route performance, failed attempts, and revenue summary.

1.5 Functional Requirements

Req. ID	Functional Requirement	Priority
FR-01	Create a parcel booking with sender/receiver, weight, service, and tracking number.	High
FR-02	Calculate and store a charge using the selected service and parcel weight.	High
FR-03	Search a parcel by its unique tracking number and show status history.	High
FR-04	Record route/branch scans and delivery outcomes with timestamps.	High
FR-05	Maintain delivery-agent, vehicle, payment, and delivery-attempt records.	Medium
FR-06	Generate operational and revenue reports using SQL.	High

FR-07	Validate inputs and handle database errors without corrupting records.	High

1.6 Entity-Relationship Diagram (ERD)

Figure 1 presents a conceptual view of the principal entities. The final approved diagram must be checked against the actual relational schema before submission. The relationship cardinalities marked with an asterisk are design assumptions and must be finalized during implementation.

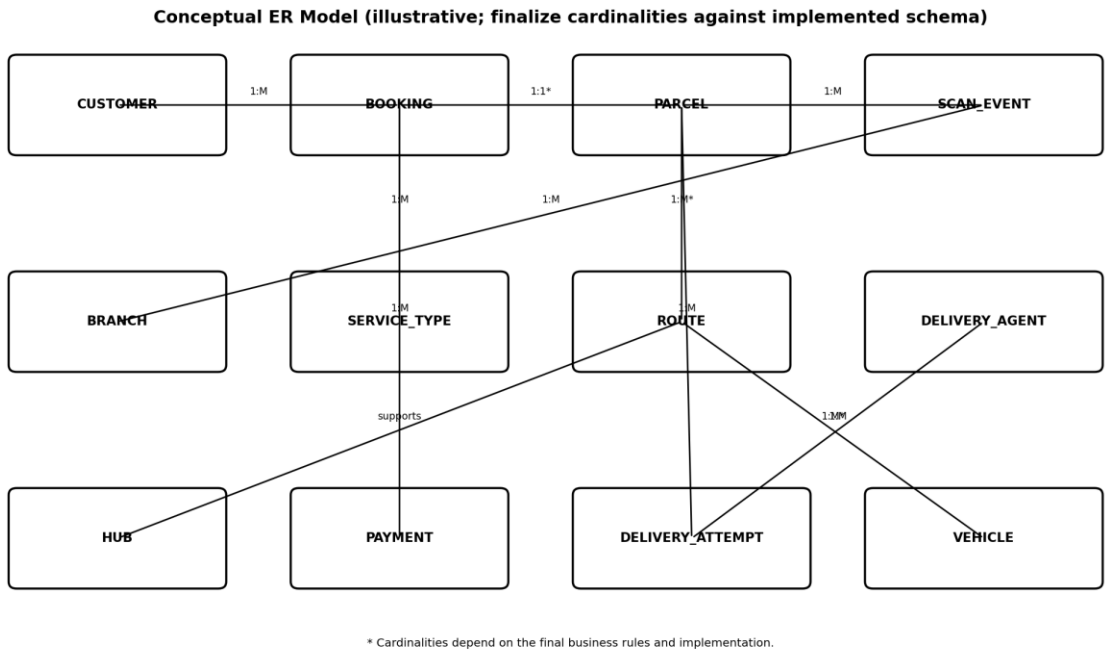


Figure 1. Entity-Relationship Diagram of the Proposed System

1.7 Entity and Relationship Summary

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
Customer	CustomerID, name, phone, email; Customer 1:M Booking	Stores sender/customer details.
Booking	BookingID, CustomerID, ServiceTypeID, booking date	Represents a booking transaction.
Parcel	ParcelID, BookingID, TrackingNumber, weight; Booking 1:1 Parcel (assumed)	Stores shipment-specific details; tracking number is unique.
Service Type	ServiceTypeID, name, base charge, rate per kg; 1:M Booking	Defines service pricing parameters.
Scan Event	ScanID, ParcelID, BranchID, status,	Maintains tracking history.

	timestamp; Parcel 1:M Scan	
Branch / Hub	BranchID/HubID, name, location	Represents operational locations.
Route / Vehicle	RouteID, origin, destination, VehicleID	Represents movement between locations.
Delivery Agent	AgentID, name, contact, BranchID	Stores delivery personnel.
Delivery Attempt	AttemptID, ParcelID, AgentID, attempt time, outcome	Records successful and failed attempts.
Charge / Payment	ChargeID, BookingID, amount; PaymentID, BookingID, amount	Stores payable charges and payment transactions.

CHAPTER 2

LOGICAL DATABASE DESIGN

2.1 Relational Schema

The following is the proposed logical schema. PK denotes primary key and FK denotes foreign key. The final schema and naming should match the actual SQL script and application.

S.No.	Table Name	Representative Attributes / Purpose
1	CUSTOMER	CustomerID (PK), Name, Phone (UNIQUE), Email, Address
2	BRANCH	BranchID (PK), BranchName, City, Address
3	HUB	HubID (PK), HubName, BranchID (FK), Location
4	SERVICE_TYPE	ServiceTypeID (PK), ServiceName (UNIQUE), BaseCharge, RatePerKg
5	BOOKING	BookingID (PK), CustomerID (FK), ServiceTypeID (FK), BookingDate
6	PARCEL	ParcelID (PK), BookingID (FK), TrackingNumber (UNIQUE), Weight, CurrentStatus
7	ROUTE	RouteID (PK), OriginBranchID (FK), DestinationBranchID (FK), VehicleID (FK)
8	VEHICLE	VehicleID (PK), RegistrationNo (UNIQUE), VehicleType
9	DELIVERY_AGENT	AgentID (PK), Name, Phone (UNIQUE), BranchID (FK)
10	SCAN_EVENT	ScanID (PK), ParcelID (FK), BranchID (FK), RouteID (FK, nullable), Status, ScanTime
11	DELIVERY_ATTEMPT	AttemptID (PK), ParcelID (FK), AgentID (FK), AttemptTime, Outcome, Remarks
12	CHARGE	ChargeID (PK), BookingID (FK), Amount, Description
13	PAYMENT	PaymentID (PK), BookingID (FK), Amount, PaymentMode, PaymentStatus, PaymentTime

2.2 Keys and Integrity Constraints

Table / Entity	Key / Constraint	Attribute(s)	Business Rule Enforced
CUSTOMER	PK / UNIQUE / NOT NULL	CustomerID / Phone / Name	Identifies each customer and prevents duplicate phone values where required.
BOOKING	PK / FK	BookingID / CustomerID, ServiceTypeID	Every booking has a valid customer and service type.
PARCEL	PK / UNIQUE / CHECK	ParcelID / TrackingNumber / Weight	Tracking number is unique; weight must be greater than zero.
SERVICE_TYPE	PK / CHECK	ServiceTypeID / BaseCharge, RatePerKg	Pricing values must be non-negative.
SCAN_EVENT	PK / FK / NOT NULL	ScanID / ParcelID, BranchID / Status, ScanTime	Scan must refer to a valid parcel and location.
PAYMENT	PK / FK / CHECK	PaymentID / BookingID / Amount	Payment must reference a booking and have a positive amount.
VEHICLE	PK / UNIQUE	VehicleID / RegistrationNo	Vehicle identifier is unique.
DELIVERY_AGENT	PK / FK / UNIQUE	AgentID / BranchID / Phone	Agent belongs to a valid branch; contact uniqueness is enforced.

2.3 Functional Dependencies

The following functional dependencies are expected in the proposed design. They assume each listed identifier uniquely determines the other attributes in its relation.

- CustomerID → Name, Phone, Email, Address
- ServiceTypeID → ServiceName, BaseCharge, RatePerKg
- BookingID → CustomerID, ServiceTypeID, BookingDate
- ParcelID → BookingID, TrackingNumber, Weight, CurrentStatus
- TrackingNumber → ParcelID, BookingID, Weight, CurrentStatus
- ScanID → ParcelID, BranchID, RouteID, Status, ScanTime
- PaymentID → BookingID, Amount, PaymentMode, PaymentStatus, PaymentTime

2.4 Normalization up to Third Normal Form (3NF)

Relation	1NF	2NF	3NF	Remarks / Decomposition

CUSTOMER	Yes, atomic fields	Yes, single-attribute PK	Yes, no transitive non-key dependency	Customer details stored separately.
BOOKING	Yes	Yes, single-attribute PK	Yes, customer/service details referenced by FK	Avoids repeating customer and service descriptions.
PARCEL	Yes	Yes	Yes, booking data not duplicated	Parcel attributes separated from booking details.
SCAN_EVENT	Yes	Yes	Yes, event details depend on ScanID	Multiple scans stored as separate rows.
PAYMENT	Yes	Yes	Yes	Each payment is a separate transaction.

The design aims for 3NF by storing facts about each entity in its own relation and using foreign keys to reference related records. The final 3NF claim should be verified against all attributes and functional dependencies in the implemented schema.

2.5 Data Dictionary

Table	Attribute	Data Type (proposed)	Key / Constraint	Description
CUSTOMER	CustomerID	INTEGER	PK	Unique customer identifier.
CUSTOMER	Name	VARCHAR(100)	NOT NULL	Customer name.
CUSTOMER	Phone	VARCHAR(20)	UNIQUE	Contact number.
BOOKING	BookingID	INTEGER	PK	Unique booking identifier.
BOOKING	CustomerID	INTEGER	FK, NOT NULL	Customer placing the booking.
BOOKING	ServiceTypeID	INTEGER	FK, NOT NULL	Selected service.
PARCEL	ParcelID	INTEGER	PK	Unique parcel identifier.
PARCEL	TrackingNumber	VARCHAR(30)	UNIQUE, NOT NULL	Customer-facing tracking code.
PARCEL	Weight	DECIMAL(8,2)	CHECK > 0	Parcel weight in kilograms.
PARCEL	CurrentStatus	VARCHAR(30)	DEFAULT	Latest parcel status.
SCAN_EVENT	ScanID	INTEGER	PK	Unique scan identifier.
SCAN_EVENT	ParcelID	INTEGER	FK, NOT NULL	Parcel associated with scan.
SCAN_EVENT	ScanTime	TIMESTAMP	NOT NULL	Time of scan event.

PAYMENT	Amount	DECIMAL(10,2)	CHECK > 0	Payment amount.

2.6 Implementation of Logical Design: DDL and Sample Data

Item	Details
DBMS	SQLite for local prototype; confirm against actual implementation.
DBMS Version	Python sqlite3 module / SQLite version: record from the execution environment.
Number of Tables	Proposed logical design: 13 tables. Starter prototype may contain fewer tables; report actual count after implementation.
Sample Data	Use synthetic customer, parcel, scan, route, delivery, and payment records. Record the actual number of rows.

Representative DDL for the current minimal Python prototype is shown below. The complete project should implement the full logical schema listed in Section 2.1; do not report all 13 tables as implemented unless they exist in the submitted database.

```
CREATE TABLE bookings (
    tracking_no TEXT PRIMARY KEY,
    sender TEXT NOT NULL,
    receiver TEXT NOT NULL,
    weight REAL NOT NULL CHECK (weight > 0),
    service TEXT NOT NULL,
    charge REAL NOT NULL CHECK (charge > 0),
    status TEXT NOT NULL DEFAULT 'Booked',
    booked_at TEXT NOT NULL
);

CREATE TABLE scans (
    scan_id INTEGER PRIMARY KEY AUTOINCREMENT,
    tracking_no TEXT NOT NULL,
    location TEXT NOT NULL,
    status TEXT NOT NULL,
    scan_time TEXT NOT NULL,
    FOREIGN KEY (tracking_no)
        REFERENCES bookings(tracking_no)
);
```

CHAPTER 3

APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

The proposed application is a local, database-backed interface for courier booking and tracking. The user enters parcel information through a form. Python validates the input, calculates an indicative charge, and uses parameterized SQL statements to store records in SQLite. Tracking search retrieves booking information and scan history. An operations screen records new scan statuses, while report screens use SQL aggregation and joins. The application must use the database as its persistent source of truth rather than relying on hard-coded records.

3.2 Development Environment and Database Connectivity

Item	Details
Programming Language	Python 3.x (record the installed version).
DBMS	SQLite for the local demonstration.
IDE / Framework / GUI Toolkit	Visual Studio Code; Streamlit.
Database Connector / Driver	Python standard-library sqlite3 module.
Operating Environment	Windows laptop; confirm exact OS version.
Repository Link	[Add GitHub URL or write “Not published”.]

The application opens a SQLite connection, enables foreign-key enforcement, and executes parameterized SQL. Parameterized statements reduce SQL injection risk and handle user-supplied values safely. Database writes that form one logical operation—such as inserting a booking and its initial scan—should be performed in one transaction so that partial records are not left behind if an operation fails.

3.3 Application Modules and Database Mapping

Module / Screen	Purpose	Tables Used	Main Operations
Parcel Booking	Capture sender, receiver, weight, and service; calculate charge.	BOOKING, PARCEL, CUSTOMER, SERVICE_TYPE, CHARGE	INSERT, SELECT

Tracking Search	Find parcel and display current status/history.	PARCEL, BOOKING, SCAN_EVENT	SELECT, JOIN, ORDER BY
Scan / Route Update	Record parcel movement at a branch or hub.	SCAN_EVENT, PARCEL, ROUTE, BRANCH, HUB	INSERT, UPDATE
Agent / Delivery	Assign agent and record delivery outcome.	DELIVERY_AGENT, DELIVERY_ATTEMPT, PARCEL	SELECT, INSERT, UPDATE
Payment	Record payment and verify outstanding balance.	PAYMENT, CHARGE, BOOKING	INSERT, SELECT, aggregate
Reports	Display status counts, delayed parcels, branch load, and revenue.	Multiple related tables	JOIN, GROUP BY, SUM, COUNT

3.4 CRUD Operations

Operation	Application Feature / Screen	Table(s) Affected	Evidence
Create	Create a parcel booking	BOOKING, PARCEL, SCAN_EVENT	Insert booking screenshot and database row.
Read / Search	Search tracking number; view parcel list	PARCEL, BOOKING, SCAN_EVENT	Search result screenshot.
Update	Record scan or delivery status	PARCEL, SCAN_EVENT	Before/after status and scan history.
Delete / Controlled Removal	Not enabled in the initial prototype; use controlled cancellation/soft-delete if required.	Relevant tables	Document policy and evidence if implemented.

3.5 Business Rules, Validation and Error Handling

Rule / Validation	How Implemented	Evidence / Message
Required names	Reject empty or whitespace-only sender/receiver input.	“Enter both sender and receiver names.”
Positive weight	UI minimum and database CHECK constraint.	Invalid weight rejected.
Unique tracking number	Generate identifier and enforce PRIMARY KEY / UNIQUE.	Duplicate key error handled.
Valid references	Foreign keys enabled for related records.	Invalid reference rejected.
No scans after delivery	Check current status before accepting another scan.	“Delivered parcels cannot receive further scans.”
Payment limit	Compare payment against outstanding balance within a transaction.	Reject payment above balance.

Database failures	Catch database exceptions and show a useful message.	Error shown without exposing sensitive details.

3.6 Reports and Database Operations

Report / Operation	SQL / Database Concept Used	Application Output
Current parcel location	Latest scan per tracking number; ORDER BY timestamp	Latest location and status.
Status summary	GROUP BY status; COUNT(*)	Number of parcels by status.
Branch workload	JOIN scan/parcel/branch; GROUP BY branch	Parcel volume by branch.
Delayed shipments	Filter parcels exceeding a defined delivery threshold	List of potentially delayed parcels.
Failed delivery attempts	Filter delivery outcome = failed	Failed attempt count and details.
Revenue by service	GROUP BY service; SUM(charge)	Bookings and charges by service.
Tracking history	JOIN booking/parcel/scan; ORDER BY scan time	Chronological scan history.

3.7 Application Screens

Insert actual screenshots captured from the running application. The report template expects evidence of data entry, search/display, update or transaction processing, and at least one report/output. Do not use illustrative images as proof of implementation.

[Insert a clear, cropped screenshot here: Figure 2. Parcel Booking Screen]

Figure 2. Parcel Booking Screen

[Insert a clear, cropped screenshot here: Figure 3. Parcel Tracking Screen]

Figure 3. Parcel Tracking Screen

[Insert a clear, cropped screenshot here: Figure 4. Delivery / Scan Update Screen]

Figure 4. Delivery / Scan Update Screen

[Insert a clear, cropped screenshot here: Figure 5. Reports and SQL Output]

Figure 5. Reports and SQL Output

3.8 Testing and Results

Execute each test on the final application and replace the “Actual Result” and “Status” entries with observed outcomes. The expected results below are test criteria, not claims that testing has already passed.

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-01	Booking	Valid sender, receiver, positive weight, valid service	Booking saved; tracking number generated.	[Record observed result]	[Pass/Fail]
TC-02	Required fields	Submit with sender or receiver blank	Validation message; no booking inserted.	[Record observed result]	[Pass/Fail]
TC-03	Weight validation	Enter zero or negative weight	Input/database validation rejects value.	[Record observed result]	[Pass/Fail]
TC-04	Tracking search	Search an existing tracking number	Matching parcel and scan history displayed.	[Record observed result]	[Pass/Fail]
TC-05	Unknown tracking	Search a nonexistent tracking number	Clear “not found” message.	[Record observed result]	[Pass/Fail]
TC-06	Delivery rule	Attempt a scan after Delivered status	Update rejected; history remains unchanged.	[Record observed result]	[Pass/Fail]
TC-07	Report query	Run status summary / JOIN query	Correct records and aggregate values displayed.	[Record observed result]	[Pass/Fail]
TC-08	Persistence	Restart app and search saved parcel	Previously committed record remains available.	[Record observed result]	[Pass/Fail]

Results and observations: [After executing the test cases, summarize the number of tests passed/failed, observed defects, and corrections made. Attach screenshots or query outputs as evidence.]

CHAPTER 4

CONCLUSION

The Courier Parcel Booking and Tracking Management System applies relational database concepts to the operational workflow of a courier service. The project identifies the entities required for booking, parcel movement, delivery, and payment, and proposes a relational schema connected through primary and foreign keys. Normalization separates data about customers, services, parcels, scans, agents, and payments to reduce unnecessary duplication and support consistent updates.

A Python application with a relational database can demonstrate booking, charge calculation, tracking lookup, status updates, and SQL-based reporting. The project also provides practical experience with constraints, parameterized queries, transactions, validation, and testing. The final conclusion should be updated to reflect the modules actually implemented and the test evidence collected.

4.1 Limitations and Future Enhancements

- The local prototype uses synthetic data and is not connected to a live courier network.
- The starter implementation may cover only a subset of the full proposed relational schema; complete the remaining entities and mappings before final submission.
- Add authentication and role-based access for booking staff, operations staff, delivery agents, and administrators.
- Integrate GPS-based tracking and notification services in a future version.
- Add stronger transaction handling, payment reconciliation, indexing, audit logs, and automated test coverage.

CHAPTER 5

REFERENCES

Use IEEE style and retain only sources actually consulted and cited in the report. The following official documentation sources are suggested; add in-text citations where their concepts or instructions are used.

[1] Python Software Foundation, “sqlite3 — DB-API 2.0 interface for SQLite databases,” Python Documentation. [Online]. Available: https://docs.python.org/3/library/sqlite3.html. Accessed: Oct. 11, 2026.

[2] Streamlit, “Streamlit documentation.” [Online]. Available: https://docs.streamlit.io/. Accessed: Oct. 11, 2026.

[3] SQLite Consortium, “SQLite Documentation.” [Online]. Available: https://www.sqlite.org/docs.html. Accessed: Oct. 11, 2026.

[4] Oracle, “MySQL 8.4 Reference Manual.” [Online]. Available: https://dev.mysql.com/doc/. Accessed: Oct. 11, 2026.

CHAPTER 6

APPENDICES

APPENDIX A — COMPLETE DATABASE SCRIPT

Attach the final SQL script containing CREATE TABLE statements for every implemented table, constraints, indexes, sample INSERT statements, views, and report queries. Suggested file name: courier_db.sql. Ensure that the script executes successfully on a clean database.

APPENDIX B — APPLICATION SOURCE CODE

Source file: app.py. Database file: courier.db (generated at runtime). Repository: [Add repository URL or submission location]. Do not include private credentials or real customer personal information.

APPENDIX C — ADDITIONAL TEST DATA AND OUTPUTS

Attach additional test data, query results, screenshots, and error-handling evidence. Use synthetic names and contact details for the academic demonstration.